

Proiectarea cu Microprocesoare
I^2C, USB

Andrei Voinescu

POLITEHNICA University of Bucharest

25 martie 2020

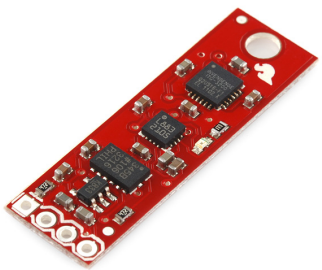
Outline

I²C

USB

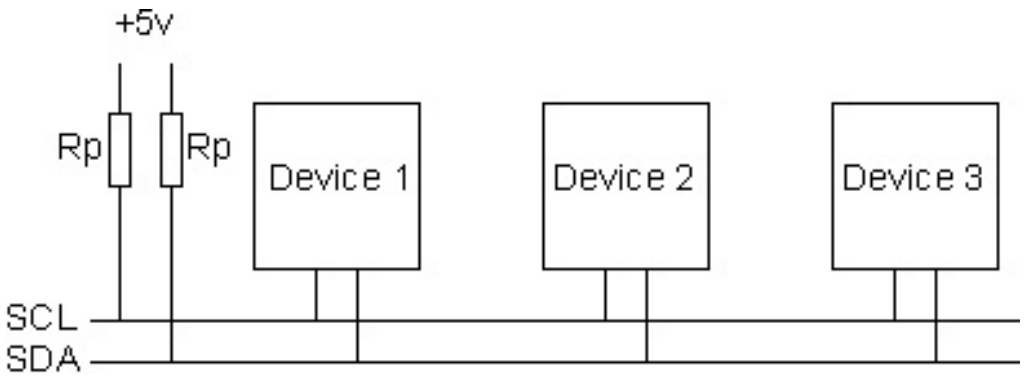
- **Inter-Integrated Circuit**, sau Two-Wire Interface (TWI)
  - Standard dezvoltat de Phillips, proprietar
- Comunicație între microcontrollere
- Comunicație cu senzori digitali

- **Inter-Integrated Circuit**, sau Two-Wire Interface (TWI)
  - Standard dezvoltat de Phillips, proprietar
- Comunicație între microcontrollere
- Comunicație cu senzori digitali
- Altele: memorii EEPROM seriale, ADC-uri, display-uri, ceasuri de timp real, date de monitorizare în PC-uri

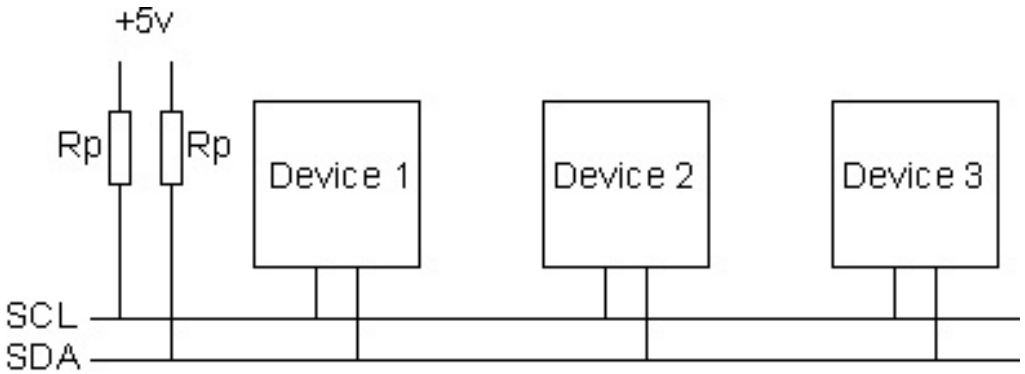




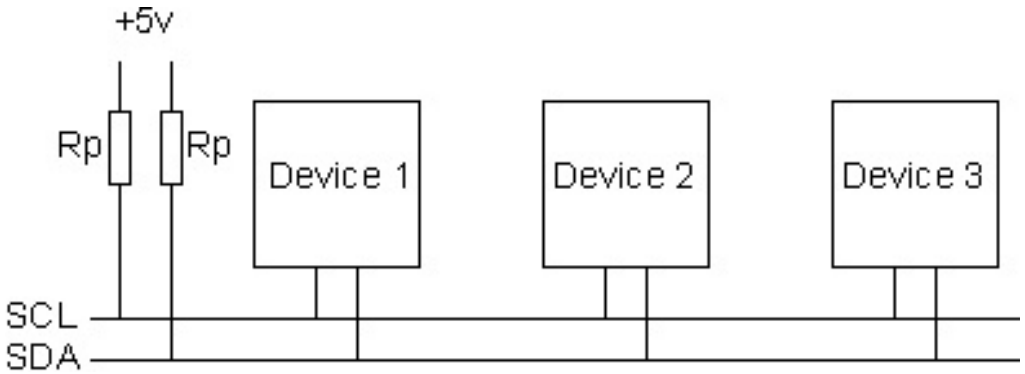




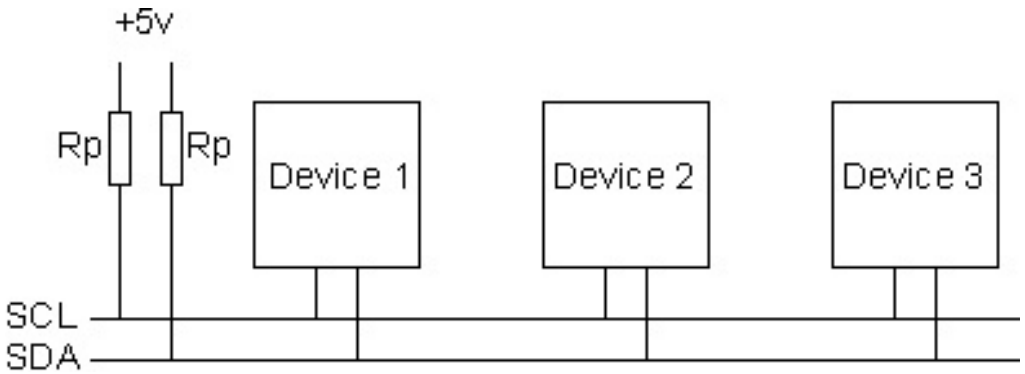
- Protocol master-slave



- Protocol master-slave
- Comunicație sincronă (semnal de ceas explicit)



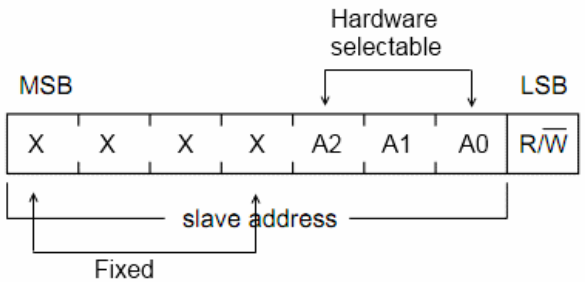
- Protocol master-slave
- Comunicație sincronă (semnal de ceas explicit)
- 2 linii de date: `SCL` și `SDA`

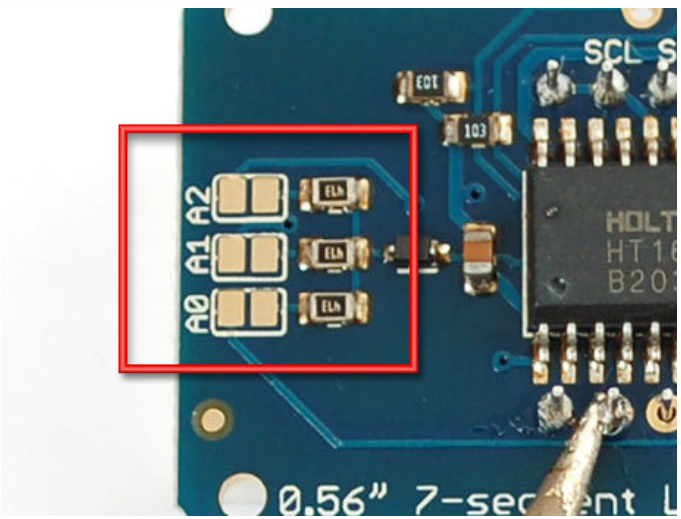


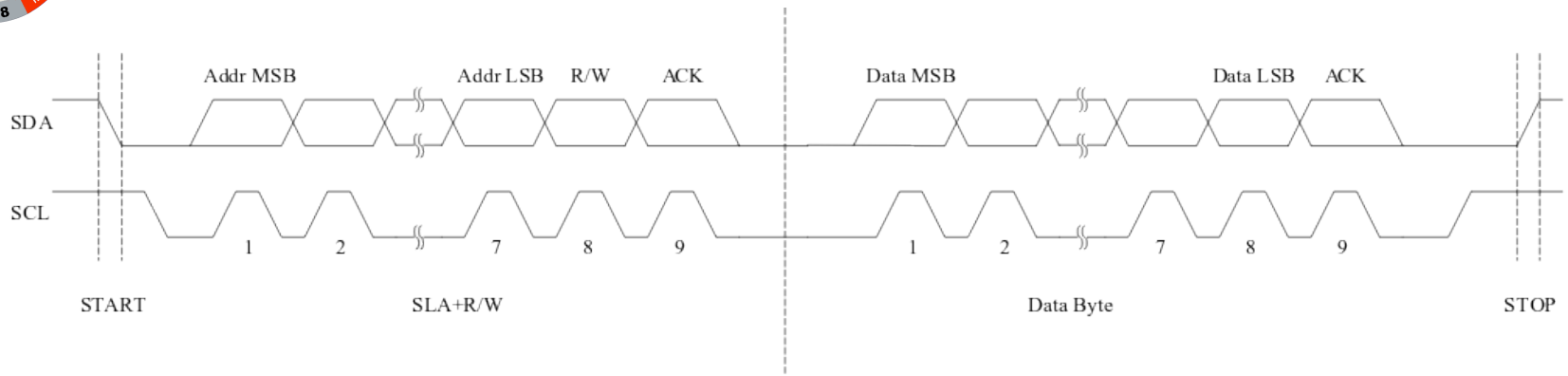
- Conexiuni open-collector $\rightarrow$ pull-up
- 1 - linie lăsată în aer, 0 - linie "trasă" la masă
- Un singur master activ

Adresele în I^{2}C

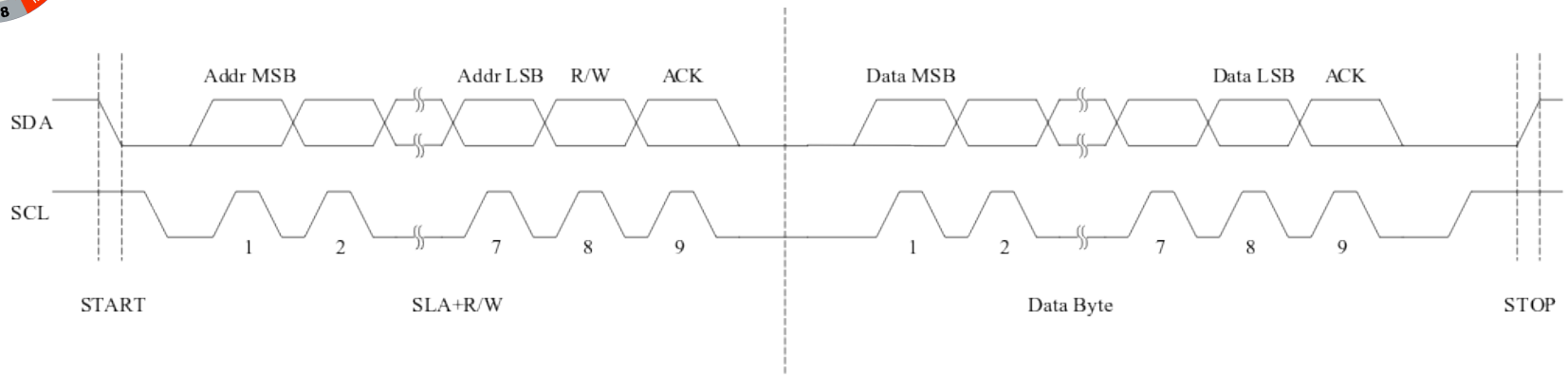
- Adrese pe 7 biți
  - Unii biți fixați
  - Alți biți configurabili hardware (imagine)
- Biții de configurare elimină conflicte între
  - Dispozitive de același fel
  - Dispozitive diferite cu aceeași adresă
- Ultimul bit (8) determină direcția comunicației



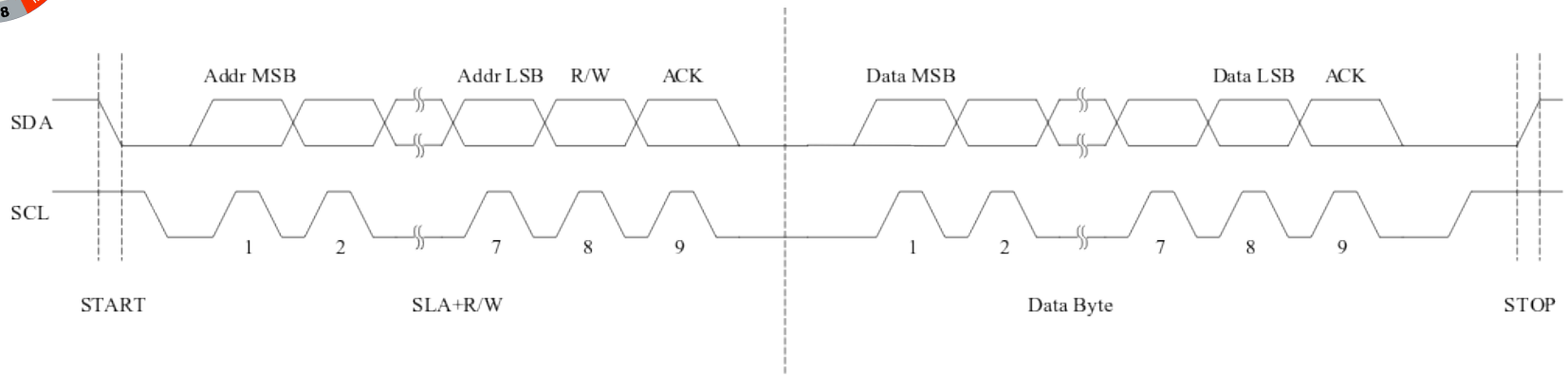




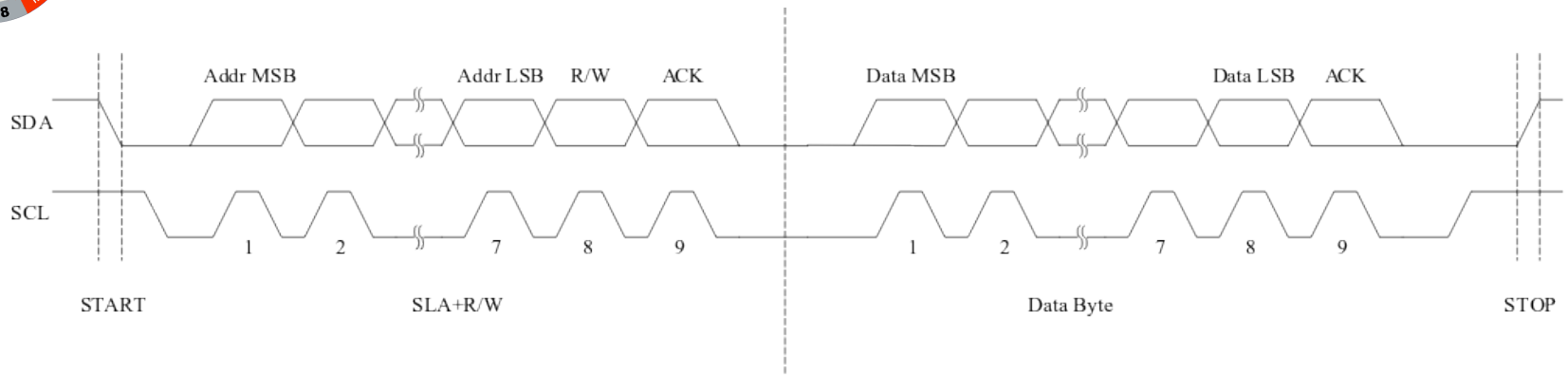
- START este 0 pe SDA când SCL este 1



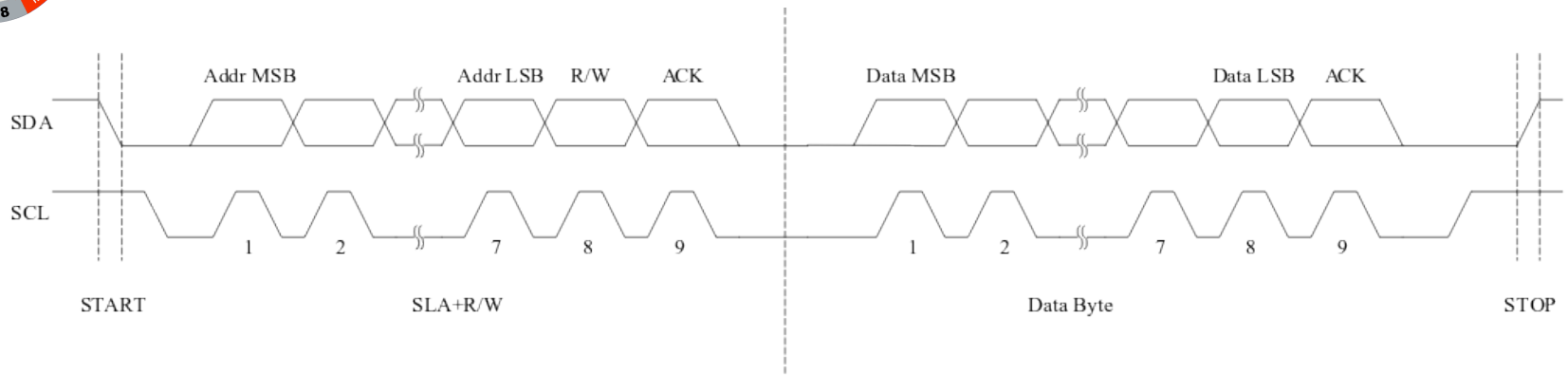
- START este 0 pe SDA când SCL este 1
- STOP este o tranziție către 1 a SDA atunci când SCL este 1



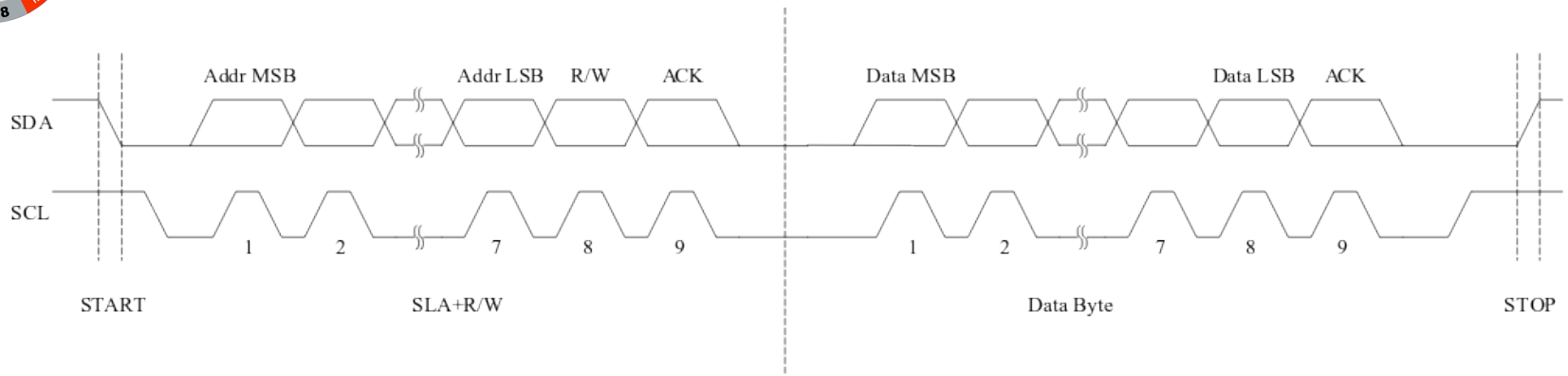
- START este 0 pe SDA când SCL este 1
- STOP este o tranziție către 1 a SDA atunci când SCL este 1
- REPEATED START == START



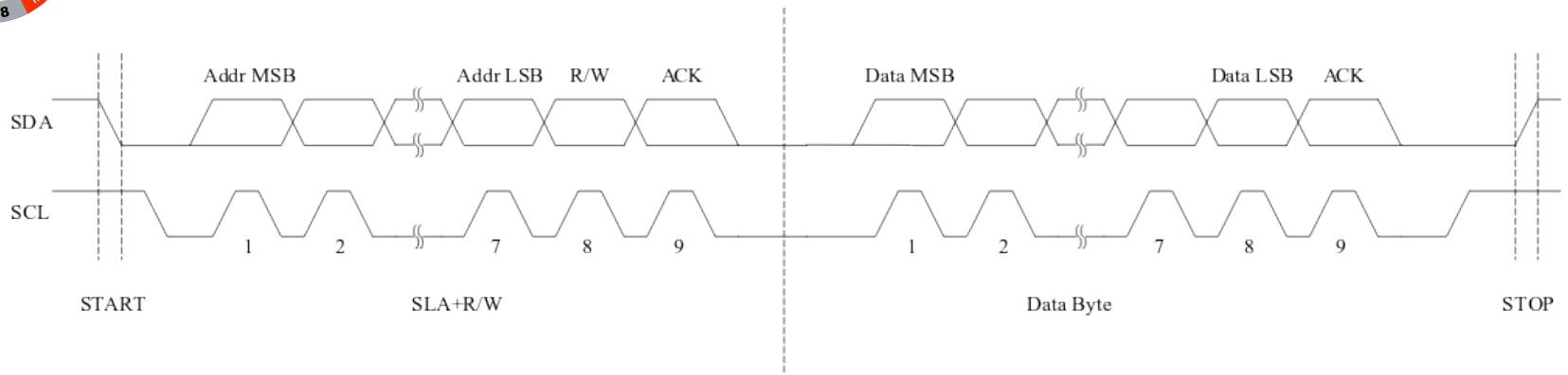
- START este 0 pe SDA când SCL este 1
- STOP este o tranziție către 1 a SDA atunci când SCL este 1
- REPEATED START == START
- ACK
  - Receiver-ul comunicației după 8 biți de date
  - Receiver ține SDA pe 0 când și SCK este pe 0



- START este 0 pe SDA când SCL este 1
- STOP este o tranziție către 1 a SDA atunci când SCL este 1
- REPEATED START == START
- ACK
  - Receiver-ul comunicației după 8 biți de date
  - Receiver ține SDA pe 0 când și SCK este pe 0
- NACK
  - Dacă nu are nimeni adresa



- START este 0 pe SDA când SCL este 1
- STOP este o tranziție către 1 a SDA atunci când SCL este 1
- REPEATED START == START
- ACK
  - Receiver-ul comunicației după 8 biți de date
  - Receiver ține SDA pe 0 când și SCK este pe 0
- NACK
  - Dacă nu are nimeni adresa
  - Trimis înainte de oprirea comunicației



- START este 0 pe SDA când SCL este 1
- STOP este o tranziție către 1 a SDA atunci când SCL este 1
- REPEATED START == START
- ACK
  - Receiver-ul comunicației după 8 biți de date
  - Receiver ține SDA pe 0 când și SCK este pe 0
- NACK
  - Dacă nu are nimeni adresa
  - Trimis înainte de oprirea comunicației
- Datele sunt valide pe frontul crescător al SCL

Exemplu semnal I^2C

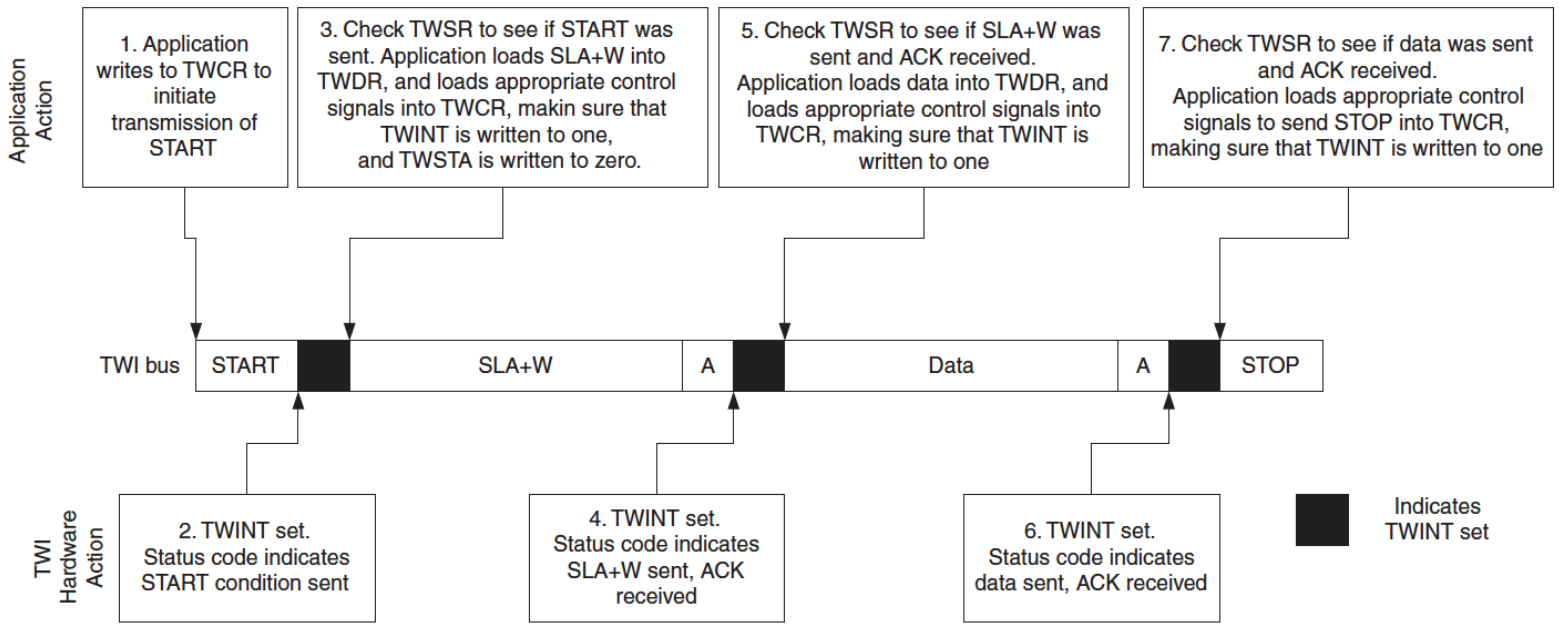


- Liniile de I^2C formează electric un ȘI logic
- Multi-master
  - Fiecare master monitorizează linia de clock (SCL)
  - În puține cazuri vor porni chiar în același timp
    - În momentul în care diferă comunicația "câștigă" cel cu 0 (își vede propriul output)

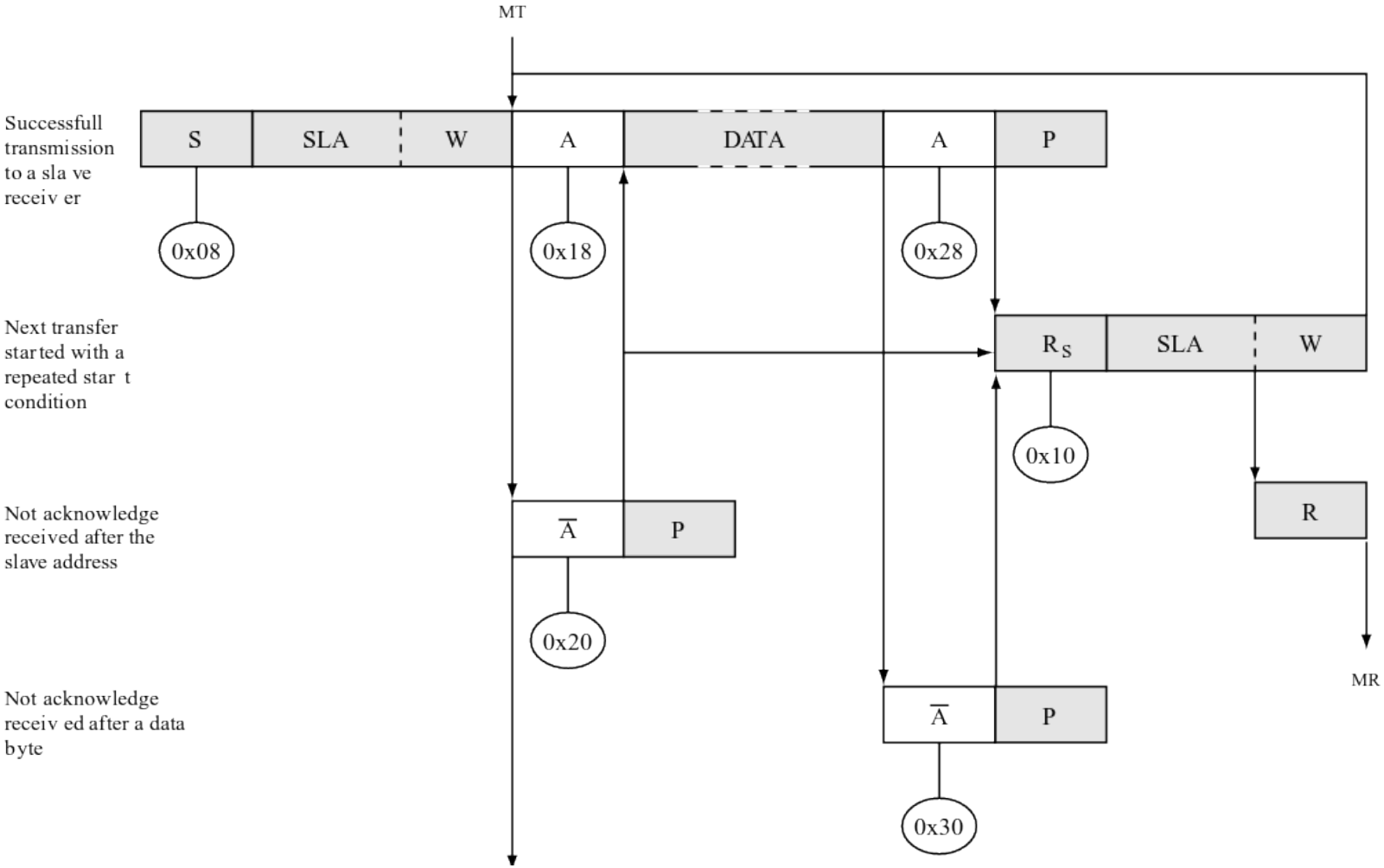
- Registre:
  - `TWCR` - flag-urile de control
  - `TWSR` - cu biți de status
  - `TWDR` - bufferul de date
  - `TWAR` - adresa dispozitivului, dacă este slave
  - `TWBR` - viteza de transmisie

- Registre:
  - `TWCR` - flag-urile de control
  - `TWSR` - cu biți de status
  - `TWDR` - bufferul de date
  - `TWAR` - adresa dispozitivului, dacă este slave
  - `TWBR` - viteza de transmisie
- Întrerupere atunci când perifericul a terminat operația curentă și așteaptă următoarea comandă

Exemplu flow transmisie



Exemplu flowchart



- mașină de stări în software



I²C - exemplu cod

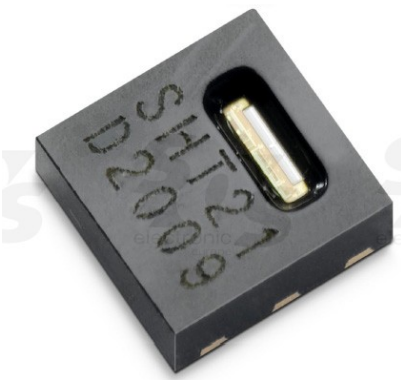
```
void i2c_writeb(uint8_t address, uint8_t value) {
  uint8_t state = STATE_START, next_state;

  TWCR0 = (1 << TWINT) | (1 << TWSTA) | (1 << TWEN);

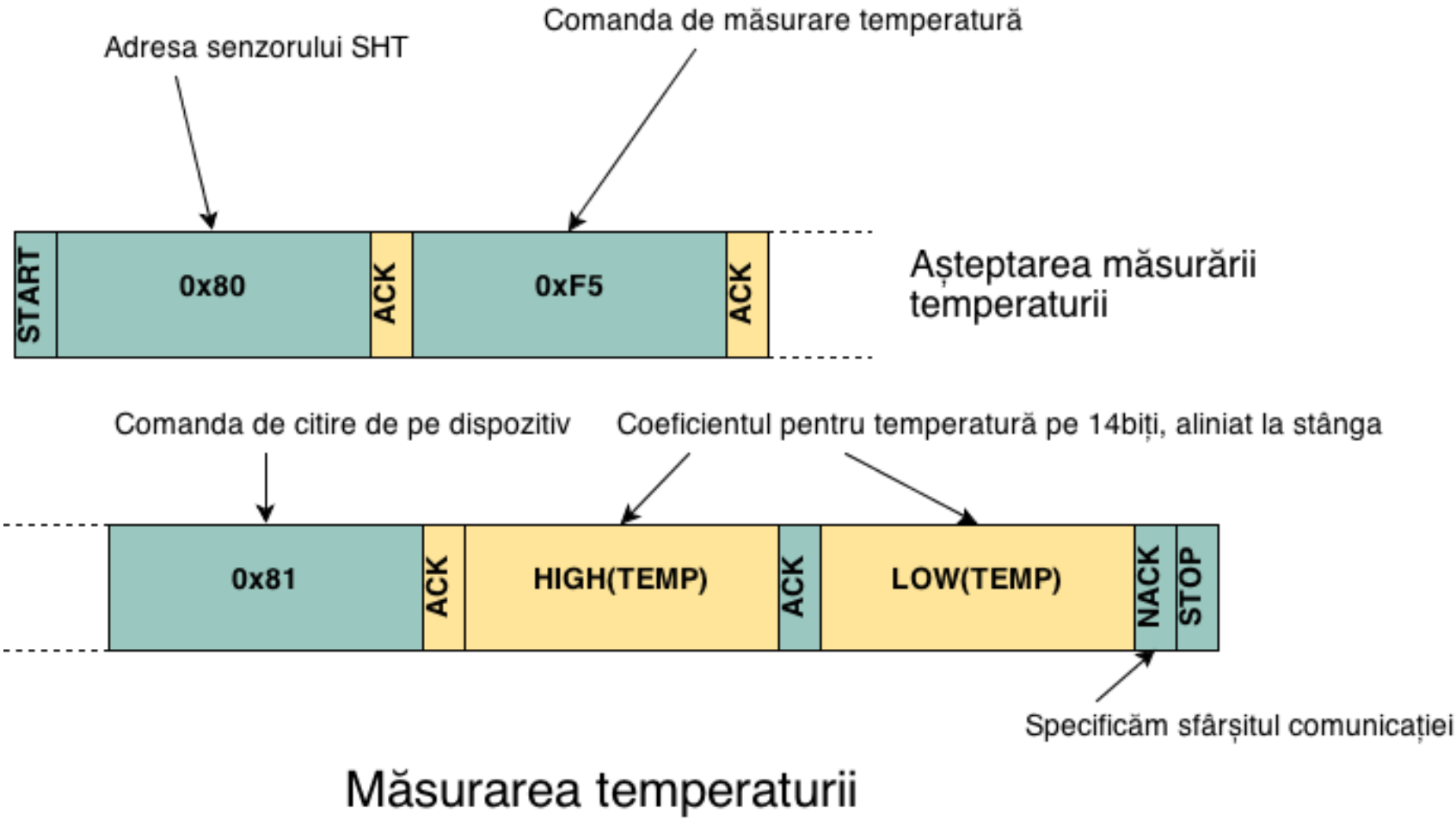
  while(1) {
    loop_until_bit_is_set(TWCR0, TWINT);
    switch(state) {
      case STATE_START:
        if ((TWSR0 & 0xf8) != STATUS_START_OK) // 0x08
          return ERROR_NO_START;
        TWDR0 = (address << 1) + SLA_W; // 0
        next_state = STATE_SLA;
        break;
      case STATE_SLA:
        if ((TWSR0 & 0xf8) != STATUS_SLA_ACK) // 0x18
          return ERROR_NO_SLA_ACK;
        TWDR0 = value;
        next_state = STATE_DATA;
        break;
      case STATE_DATA:
        if ((TWSR0 & 0xf8) != STATUS_DATA_ACK) // 0x18
          return ERROR_NO_DATA_ACK;
        next_state = STATE_STOP;
        TWCR0 = (1 << TWINT) | (1 << TWEN) | (1 << TWSTO);
        return OK;
    }

    TWCR0 = (1 << TWINT) | (1 << TWEN);
    state = next_state;
  }
}
```

- **SHT21**
  - Senzor auto-calibrat de temperatură și umiditate
  - Adresa I²C pe 7 biți este 0x40
  - Măsurarea temperaturii durează cam 100ms (așteptare sau clock stretching)
    - Așteptarea presupune ca master-ul să știe cât este delay-ul între pornirea măsurătorii și terminarea ei, pentru a cere rezultatul după terminare
    - Clock stretching este metoda non-standard prin care slave-ul prelungește un bit 0 de pe SCK pentru a ține ocupat masterul și magistrala

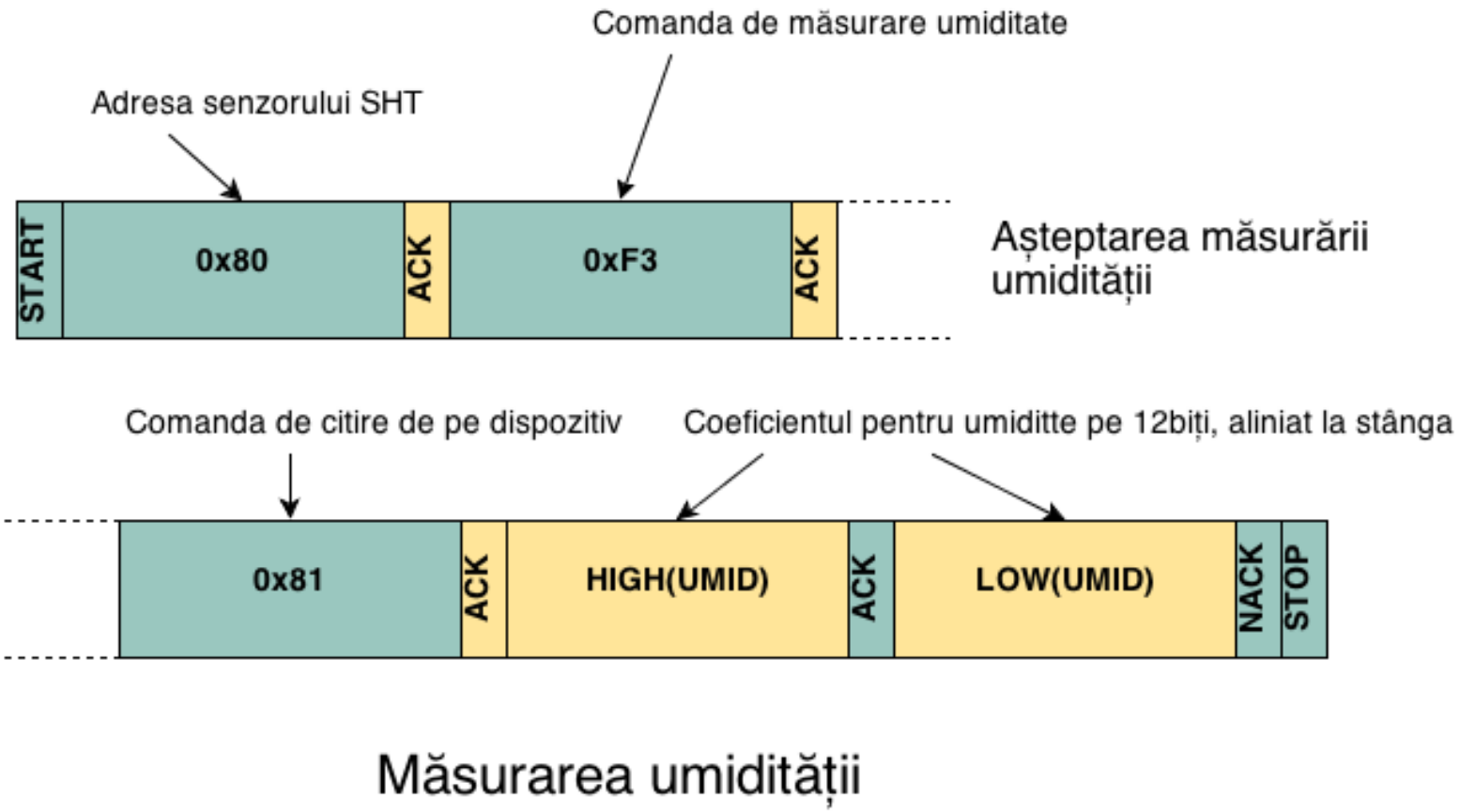


Exemplu comunicație I^{2}C (II)

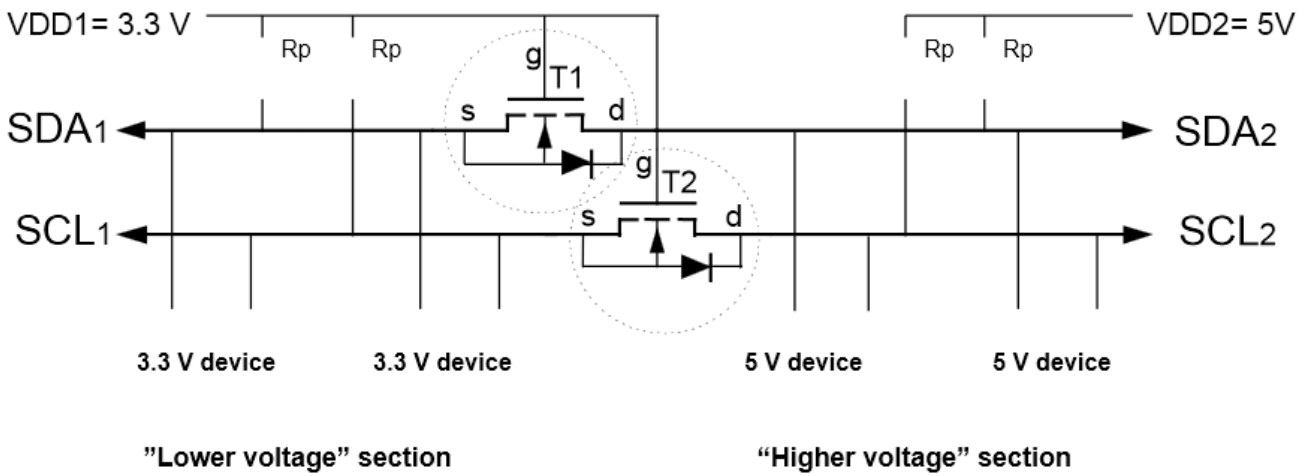


Măsurarea temperaturii

Exemplu comunicație I^{2}C (III)



Ajustare nivele de tensiune pe I^{2}C



- Dacă avem dispozitive cu nivele logice diferite?
- Adaptarea nu se poate face cu divizor rezistiv
- Schema de mai sus garantează adaptarea nivelelor logice
  - În repaus este 1 logic pe ambele părți, MOSFET-ul închis
  - În 0 logic pe tensiunea mică va apărea un $v_{gs} >$ prag, deci MOSFET-ul va fi deschis $\rightarrow$ 0 logic pe tensiunea mare
  - Cu 0 logic pe tensiunea mare va fi leakage current pe dioda de protecție $\rightarrow$ $v_{gs} >$ prag



- SMBus (System Management Bus)

- SMBus (System Management Bus)
  - Încărcarea bateriei, temperaturi interne, ventilatoare
- I^3C (Improved I^2C)

- SMBus (System Management Bus)
  - Încărcarea bateriei, temperaturi interne, ventilatoare
- I^3C (Improved I^2C)
  - Standard apărut 2017
  - Mai eficient energetic
  - Viteze de transfer mai mari decât I^2C
  - Feature-uri avansate



Concluzii I^2C

- Serială cu puține fire (2)



- Serială cu puține fire (2)
- Adresarea se face prin firul de date



- Serială cu puține fire (2)
- Adresarea se face prin firul de date
- Suport multi-master



- Serială cu puține fire (2)
- Adresarea se face prin firul de date
- Suport multi-master
- Viteze relativ mici (400kHz vs. 8MHz la SPI)



Concluzii I^2C

- Serială cu puține fire (2)
- Adresarea se face prin firul de date
- Suport multi-master
- Viteze relativ mici (400kHz vs. 8MHz la SPI)
- Necesită software mai complex



Outline

I^2C

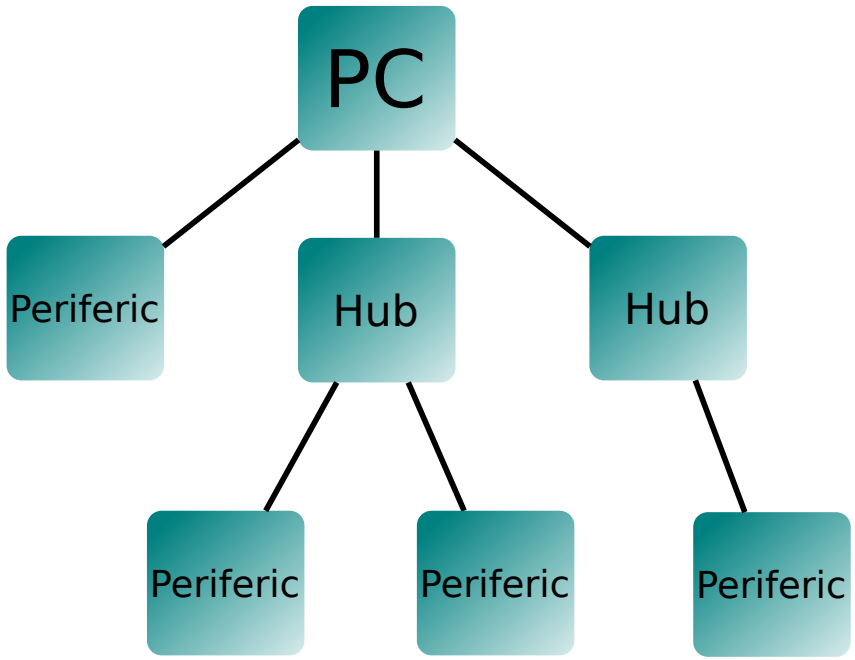
USB

- Standard definit în anii '90
- Unifică interfețe și alimentări
- USB1.0 1996
- USB2.0 2000
  - Viteze de transfer îmbunătățite (480Mbps)
  - Conectori mini
  - OTG în 2006
- USB3.0 2011
  - 5Gbps
  - Comunicație full-duplex
- USB3.1 2014
  - 10Gbps
  - Conector unic, simetric(!)

- Număr redus de conectori (A/B/microAB/miniAB/C)
- Fără cablu de alimentare
- Număr de porturi extensibil
- Rate de transfer bune
- Hot plugging
- Fără configurări manuale

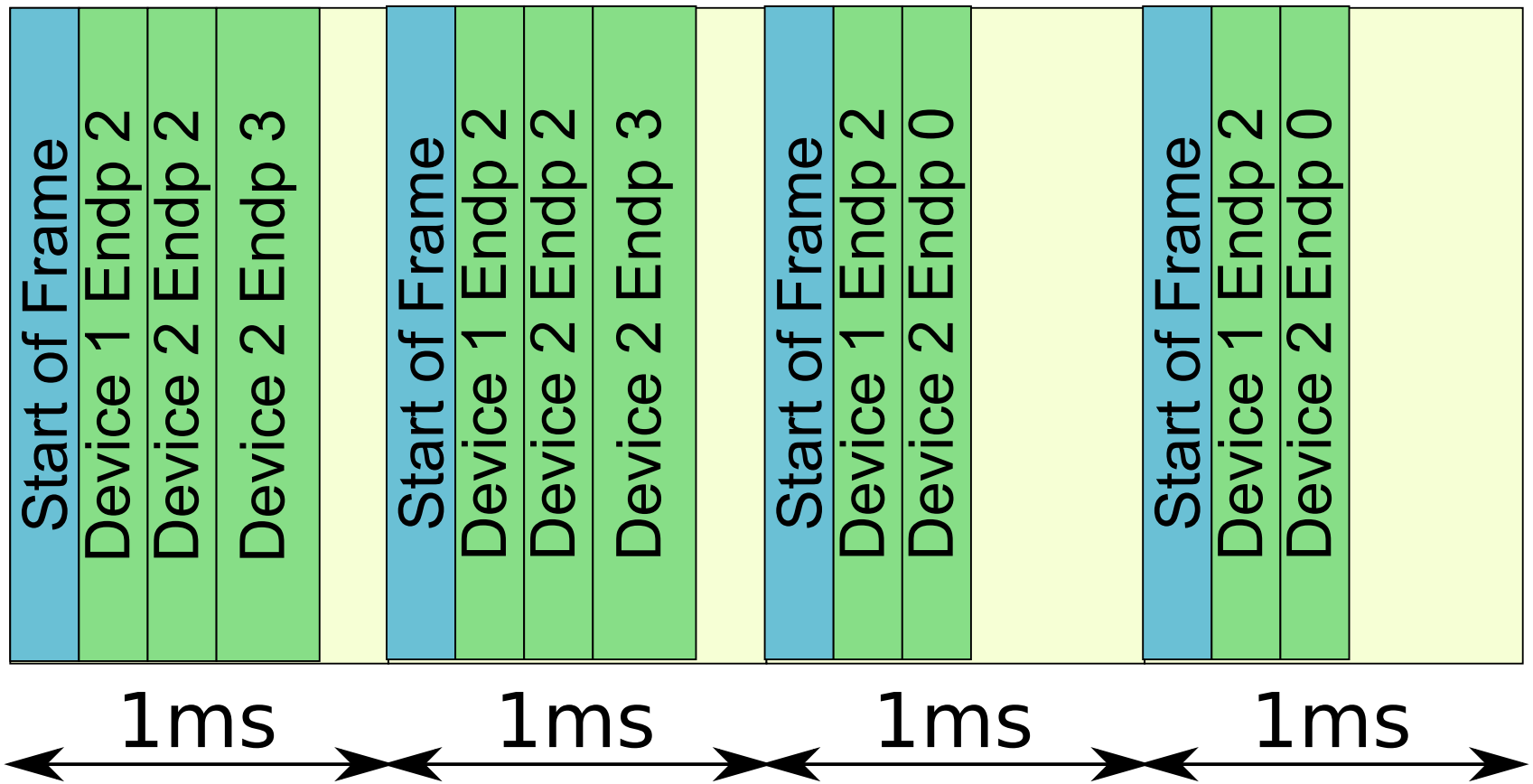
- Avantaje
  - Fără alimentare separată
  - Interfațare unică pentru dispozitive diferite
    - Se pot implementa chip-uri dedicate
  - Suport software în SO
  - Documentație din partea USB-IF
- Dezavantaje
  - Complexitate mărită (de ex. față de rs232)

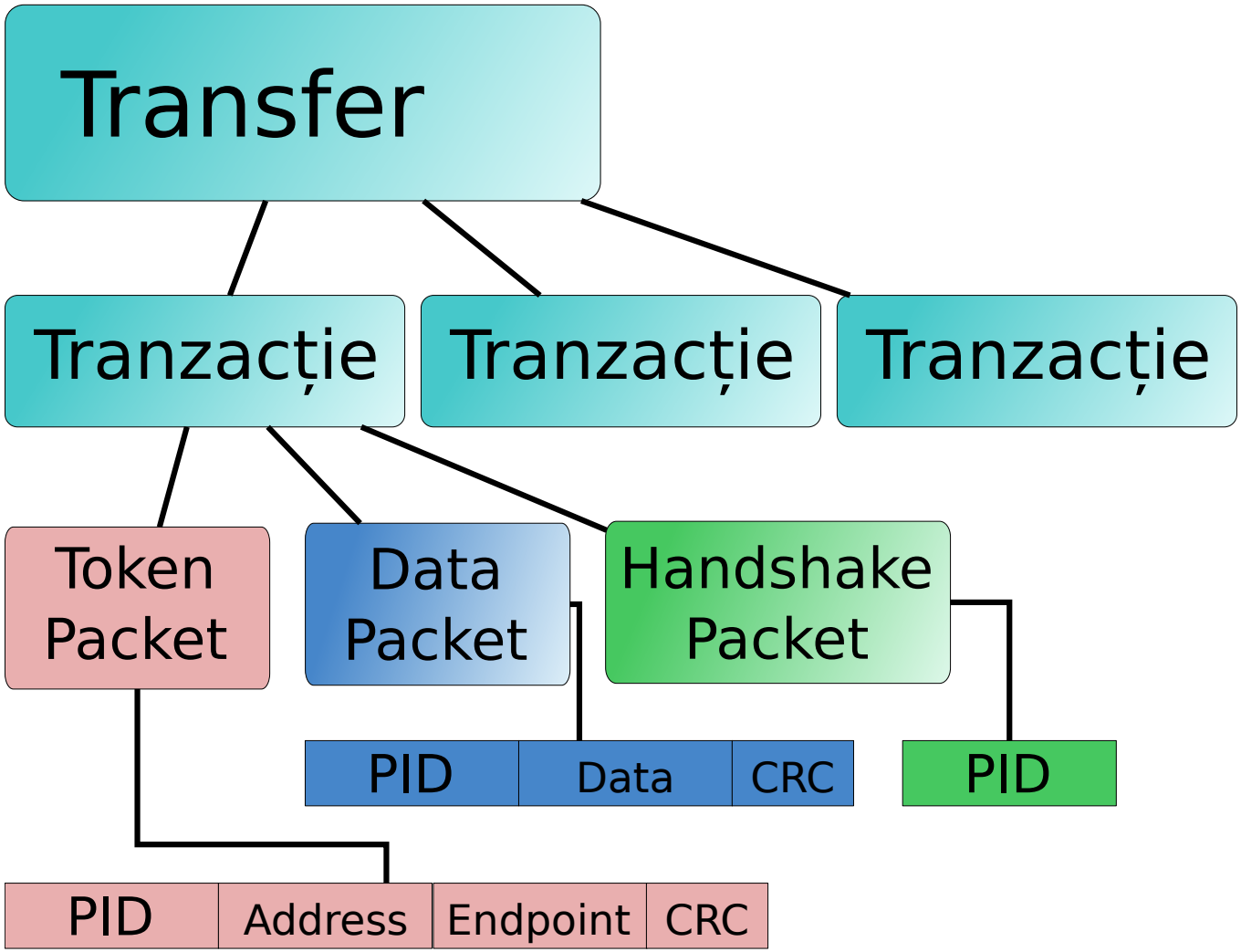
- Arbore:
  - Perifericele - frunze
  - Host,hub-uri - noduri
- Comunicație:
  - Half-duplex
  - Condiționată de host
  - Numai între host și un dispozitiv



- Cunoaște toate dispozitivele
  - capabilitățile
  - nevoile de transfer de date
  - cerințele de consum
- Administrează comunicația
  - Alocă lățime de bandă pentru anumite dispozitive
  - Anunță comunicația dispozitiv→host
  - Interoghează dispozitive

- Responsabil să se identifice host-ului
- Canale de comunicare (endpoints $\leftarrow$ ports)
- Recunoaște comunicația adresată lui
- Răspunde la cererile host-ului
- Menține consumul în limitele declarate
  - Reduce consumul când bus-ul este în SLEEP
- Comunică cu host-ul atunci când acesta cere


Start of Frame
Device 1 Endp 2
Device 2 Endp 2
Device 2 Endp 3
Start of Frame
Device 1 Endp 2
Device 2 Endp 2
Device 2 Endp 3
Start of Frame
Device 1 Endp 2
Device 2 Endp 0
Start of Frame
Device 1 Endp 2
Device 2 Endp 0
1ms
1ms
1ms
1ms


Transfer
Tranzacție
Tranzacție
Tranzacție
Token Packet
Data Packet
Handshake Packet
PID
Data
CRC
PID
PID
Address
Endpoint
CRC

Tip	**Control**	**Interrupt**	**Isochronous**	**Bulk**
Folosire	Identificare Configurare	Mouse Keyboard	Webcam	Storage
Obligatoriu	Da	Nu	Nu	Nu
Low-speed	Posibil	Posibil	Nu	Nu
Transfer	Medie	Mică	Medie	Mare
Trafic	Min 20%	-	Max 80%	-



Endpoints

- Analog porturilor de la socket
- Endpoint-ul este unidirecțional - poate fi IN sau OUT
- Poate avea un singur tip de transfer $\rightarrow$ tip endpoint
- Endpoint 0 - de control
- Fiecare funcție a dispozitivului - endpoints
- `/sys/bus/usb/devices` conține toate informațiile
- `$ cat /sys/bus/usb/devices/usb1/ep_00/type`



- Procedură de înregistrare a unui device la host
- Dispozitivul este atașat fizic

- Procedură de înregistrare a unui device la host
- Dispozitivul este atașat fizic
- Hub-ul
  - detectează dispozitivul
  - informează host-ul
  - detectează viteza cu care se poate comunica
  - resetează device-ul
  - Host-ul află dacă device-ul este high-speed
  - conectează dispozitivul la bus

- Procedură de înregistrare a unui device la host
- Dispozitivul este atașat fizic
- Hub-ul
  - detectează dispozitivul
  - informează host-ul
  - detectează viteza cu care se poate comunica
  - resetează device-ul
  - Host-ul află dacă device-ul este high-speed
  - conectează dispozitivul la bus
- Host-ul
  - alocă o adresă
  - cere descriptorii device-ului
  - selectează un driver potrivit
- Driver-ul selectează o configurație din cele disponibile

- Procedură de înregistrare a unui device la host
- Dispozitivul este atașat fizic
- Hub-ul
  - detectează dispozitivul
  - informează host-ul
  - detectează viteza cu care se poate comunica
  - resetează device-ul
  - Host-ul află dacă device-ul este high-speed
  - conectează dispozitivul la bus
- Host-ul
  - alocă o adresă
  - cere descriptorii device-ului
  - selectează un driver potrivit
- Driver-ul selectează o configurație din cele disponibile
- Încercați să urmăriți acasă cu Wireshark!

- Descriptorii sunt structuri care prezintă capabilitățile și configurările unui dispozitiv către host
- Device descriptor
  - Vendor ID, Product ID, link către string descriptors cu numele
  - Nivelul de compatibilitate USB
  - Numărul de configurații posibile
- Configuration Descriptor
  - Descrie un set de funcționalități care pot fi active simultan
  - Configurația curentă este selectată de Host bazându-se pe compatibilitate
- Interface Descriptor
- Endpoint Descriptor
- String Descriptor



Exemplu de Descriptor

Type	Field	Value
uint8_t	bLength	18
uint8_t	bDescriptorType	0x01 (Device descriptor)
uint16_t	bscUSB	0x0200
uint8_t	bDeviceClass	0x00 (Composite)
uint8_t	bDeviceSubclass	0x00 (No specific subclass)
uint8_t	bDeviceProtocol	0x00 (No specific protocol)
uint8_t	bMaxPacketSize	64
uint8_t	idVendor	0x03eb (VID of Atmel)
uint8_t	idProduct	0x2018
uint16_t	bcdDevice	0x1000
uint8_t	iManufacturer	0 (no manufacturer string)
uint8_t	...	...



- Communication Device Class
  - Abstract Control Model
    - `/dev/ttyACM0`



- Communication Device Class
  - Abstract Control Model
    - `/dev/ttyACM0`
- Mass Storage - stick-uri de memorie



Exemple de clase

- Communication Device Class
  - Abstract Control Model
    - `/dev/ttyACM0`
- Mass Storage - stick-uri de memorie
- Monitor



Exemple de clase

- Communication Device Class
  - Abstract Control Model
    - `/dev/ttyACM0`
- Mass Storage - stick-uri de memorie
- Monitor
- Video class - webcams



Exemple de clase

- Communication Device Class
  - Abstract Control Model
    - `/dev/ttyACM0`
- Mass Storage - stick-uri de memorie
- Monitor
- Video class - webcams
- Audio class - microfoane



Exemple de clase

- Communication Device Class
  - Abstract Control Model
    - `/dev/ttyACM0`
- Mass Storage - stick-uri de memorie
- Monitor
- Video class - webcams
- Audio class - microfoane
- Printer

- ATMega32U4 este printre puținele chip-uri de AVR care au periferic USB, niciunul dintre controlllere care se găsesc în pachet DIP nu au
- Challenges:
  - Funcționarea full-speed necesită un ceas de 48MHz care este format din frecvența inferioară pe care o are controller-ul
  - Transmisia la viteze superioare necesită mult buffering $\rightarrow$ memorie dual-port
  - Endpoint-uri limitate ca memorie și număr, deoarece orice endpoint necesită un buffer
    - 1 control endpoint de 64 bytes
    - 1 endpoint de 256 bytes
    - 5 endpointuride 64 bytes

- Dacă este prezent perifericul de USB
  - Management alimentare
  - Management conectare la bus (+enumerare)
  - Power management periferic
  - Management endpoint-uri
  - Management-ul memoriei alocate endpoint-urilor
- 7 + 8 posibile întreruperi * 7 endpoints = 63
- 20 de registre I/O pentru control/status
- Biblioteci:
  - LUFA - `http://www.fourwalledcubicle.com/files/LUFA/Doc/120219/html/index.html`
  - Atmel - `http://atmel.com/dyn/products/app_notes.asp?family_id=607#USB`



USB - exemplu cod LUFA

```
/* Exemplu HID - emulator mouse/tastatura */

// (...) Structuri configurare

void main()
{
  SetupHardware();
  GlobalInterruptEnable();

  for (;;)
  {
    HID_Device_USBTask(&Generic_HID_Interface);
    USB_USBTask();
  }
}

// (...) SetupHardware (apeleaza si USB_Init)

// (...) Event callbacks (e.g. EVENT_USB_Device_Disconnect

CALLBACK_HID_Device_CreateReport(USB_ClassInfo_HID_Device_t* const HIDInterfaceInfo,
                                 uint8_t* const ReportID,
                                 const uint8_t ReportType,
                                 void* ReportData,
                                 uint16_t* const ReportSize)
{
  /* TODO date puse in ReportData si ReportSize */
}

// (...) Callback host->device CALLBACK_HID_Device_ProcessHIDReport
```

- Pe ATMega324(p)a putem folosi USB software:
  - V-USB - `https://www.obdev.at/products/vusb/index.html`
  - Implementează funcționalitate de low-speed device
    - USB HID - mouse/tastatură/joystick
    - USB ACM - serială
    - Clase custom
  - Folosește `PD2` și `PD3` (external interrupts)

```c
/* Exemplu HID - emulator mouse/tastatura */

// (...) Structuri configurare ex: descriptorii

void main()
{
  (...) // initializare

  for(;;)
  {
    usbPoll();

    if(usbInterruptIsReady())
    {
      /* TODO pus date in reportBuffer */

      usbSetInterrupt((void *) &reportBuffer, sizeof(reportBuffer));
    }
  }
}
```

Exemplu disponibil pe `https://github.com/obdev/v-usb/blob/master/examples/hid-mouse/firmware/main.c`



- Serială cu puține fire
- Hot-plugging
- Gestionare dificilă pe un control cu putere de procesare limitată
- Nu există periferic USB pe ATMega324a
- Folosim biblioteci pur software sau peste periferic